Tighter Approximation Schemes for Chain-Graph Logical Credal Networks

IBM Research

Abstract

A chain-graph Logical Credal Network (LCN) admits a factorization that captures its Local Markov Condition (LMC) independence assumptions. The current `cn/` pipeline exploits this by compiling the LCN into a *credal network*: a directed graph whose local credal sets $K(\text{child} \mid \text{parents})$ are interval-valued and computed per family, then performing posterior inference (Credal Variable Elimination) over the *strong extension* of that credal network. The strong extension is generally a *proper superset* of the LCN's distribution set, so the computed posterior marginal bounds are looser than the exact LCN bounds. This note dissects the two distinct sources of this looseness on a running example (the `alarm` network), proposes five approximation schemes that produce tighter outer approximations—hence tighter posterior bounds—illustrates each on the example, explains how the existing inference algorithms (Credal VE, Interval BP, ApproxLP, Credal CTE, Credal IJGP) all inherit the tightening, and gives a validation strategy based on certified local solves and the existing LMC factorization verifier.

1 Background and notation

Let an LCN over propositional atoms $\mathbf{V} = \{V_1, \ldots, V_n\}$ define a set of joint distributions

$$\mathcal{D}_{\text{LCN}} \;=\; \big\{\, P \in \Delta(2^{\mathbf{V}}) \;:\; P \text{ satisfies every sentence } s \,\big\},$$

where each sentence s is a Type-1 constraint $\ell \le P(\varphi) \le u$ or a Type-2 constraint $\ell \le P(\varphi \mid \psi) \le u$. When the LCN's structure graph is a chain graph, the joint factorizes over the families (child + parents) of the simplified structure graph,

$$P(\mathbf{V}) \;=\; \prod_{v} P\big(\text{ch}_v \mid \text{pa}_v\big), \tag{1}$$

and this factorization reproduces exactly the LMC independencies (cf. `docs/ISIPTA2025_LCNs.pdf` Sec. 3, `docs/LCN_IJAR_Revised.pdf`).

The `cn/` pipeline realizes (1) in three stages, each of which is implemented in a distinct module:

1. **Symbolic factorization** (`cn/factorization.py`): enumerate the families and record, per family, the child, parents, flattened atom scope and the LCN sentences *attached* to that family.

2. **Local credal sets** (`cn/local_credal_sets.py`): for each family and each interpretation of its scope, solve a (fractional) optimization problem to obtain the lower/upper bound $[\underline{p}, \overline{p}]$ of the conditional $P(\text{ch}_v = c \mid \text{pa}_v = \pi)$. The method axis carries exactly two settings (Section 1.1): `method="linear"` (the baseline) treats each family *in isolation*, and `method="linear-tight"` solves the same family against every in-scope LCN sentence plus the scope-restricted LMC equalities (scheme D1 of Section 4).

3. **Inference over the strong extension** (`cn/vertices.py`, `cn/cve.py`): enumerate the extreme points of each local credal set and run Credal Variable Elimination, which optimizes over the strong extension—each local credal set, for each parent configuration, contributes a freely and independently chosen extreme point.

1.1 Build API: two methods, two orthogonal knobs

The schemes below are exposed through the existing build entry points (`CredalNetwork.from_lcn`, `CredalNetworkVertices.from_lcn`) without multiplying flags. The previous `method="nlp"` option (which added only *pairwise* parent marginal independence) is **removed**: it is a strict half-measure for source A, dominated by D1 below, and its bilinear program added solver fragility for little tightening. The surface is therefore:

- `method` $\in$ {`"linear"`, `"linear-tight"`}. `"linear"` is today's per-family-in-isolation baseline (unchanged). `"linear-tight"` is scheme D1: each family is solved against every LCN sentence whose atom scope lies inside the family scope, plus the scope-restricted LMC bilinear equalities.
- `solver` $\in$ {`"ipopt"`, `"scip"`} (already present). On `"linear-tight"`, `solver="scip"` *is* scheme D3: it solves the enriched local program to certified global optimality; `"ipopt"` is the fast, hardened (multi-restart + SLSQP-fallback) variant of the same program.
- `merge_budget` k (new): scheme D2. A pre-factorization pass merges adjacent families whose combined scope is $\leq k$ before `ChainGraphFactorization.build`; k=1 is no merge (today), k= full scope is exact. Orthogonal to `method` and `solver`.

Thus D1 is the *core* of `"linear-tight"`, while D2 (k) and D3 (`solver`) are independent cost-vs-tightness dials rather than separate method names. (This `cn/` factorization `method` is unrelated to the identically named parameters of `ibp.py` and `map/exact_map.py`.)

2 Running example: the `alarm` network

We use `examples/alarm.lcn` throughout. It has five atoms $\{A, B, C, D, E\}$ and five sentences:

```
s1: 0.1  <= P(B)                <= 0.2
s2: 0.05 <= P(E)                <= 0.1
s3: 0.6  <= P(A | (B or E))     <= 0.9
s4: 0.5  <= P(not(C xor D) | A) <= 0.7   ; True
s5: 0.05 <= P(A)                <= 0.1
```

Its structure is a chain graph. The simplified structure graph has the compound node C–D (because $s4$ ties C and D together through $C \oplus D$), so the families and the LMC independencies the pipeline derives are:

Family	Attached sentences
$P(A \mid B, E)$	$s1, s2, s3, s5$
$P(B)$	$s1$
$P(E)$	$s2$
$P(C{-}D \mid A)$	$s4, s5$

LMC independencies
$(B \perp E)$
$(C \perp B, E \mid D, A)$
$(D \perp B, E \mid C, A)$

The factorization is $P(A \mid B, E)\, P(C{-}D \mid A)\, P(B)\, P(E)$.

Local credal sets computed today (`method="linear"`). Solving each family in isolation gives the conditionals below (lower/upper, rounded). Note the many vacuous $[0, 1]$ entries—these are exactly where gap A bites.

$P(A \mid B, E)$		$P(C-D \mid A)$	
A=0, B=0, E=0	[0.956, 1.0]	C=0, D=0, A=1	[0, 0.7]
A=1, B=0, E=0	[0, 0.044]	C=0, D=1, A=1	[0, 0.5]
A=0, B=0, E=1	[0, 1.0]	C=1, D=0, A=1	[0, 0.5]
A=1, B=0, E=1	[0, 1.0]	C=1, D=1, A=1	[0, 0.7]
$\cdots$ (rows with B=1 or E=1)	[0, 1.0]	$\cdots$ (A=0 rows)	[0, 1.0]

Two phenomena stand out and recur in the discussion below:

- $s3$ **only constrains the** $(B \vee E)$ **corner.** $s3$ bounds $P(A \mid B \vee E)$, i.e. the conditional given the *disjunction*. It says nothing directly about $P(A \mid B{=}0, E{=}1)$ or $P(A \mid B{=}1, E{=}0)$ individually, so those parent configurations come back vacuous $[0, 1]$—even though, jointly with $s5$ ($P(A) \in [0.05, 0.1]$) and the marginals $s1, s2$, the LCN pins them far tighter.
- $s4$ **constrains a sum, not a single state.** $s4$ bounds $P(\neg(C \oplus D) \mid A){=}P(C{=}D \mid A)$, i.e. the sum of the two diagonal states $P(C{=}0, D{=}0 \mid A) + P(C{=}1, D{=}1 \mid A)$. The isolated family LP can therefore push any single state to its extreme (hence $[0, 0.7]$ / $[0, 0.5]$), and for A=0 there is no constraint at all, giving $[0, 1]$.

This is the concrete object the schemes of Section 4 tighten.

3 Two sources of looseness

Write $\widetilde{\mathcal{D}}$ for the joint credal set actually optimized by CVE. We have $\mathcal{D}_{\text{LCN}} \subseteq \widetilde{\mathcal{D}}$, and the inclusion is typically strict for *two independent reasons*.

3.1 (A) Local-set widening

For family v the pipeline computes

$$K_v^{\text{local}} = \Big\{ P(\text{ch}_v \mid \text{pa}_v) \ : \ \text{the sentences } \textit{attached to family } v \text{ hold} \Big\},$$

because `_linear_rows_and_obj` loads only `family["sentences"]`. The true projection of the LCN onto that family is

$$K_v^{\text{true}} = \Big\{ P(\text{ch}_v \mid \text{pa}_v) \ : \ \exists\, P \in \mathcal{D}_{\text{LCN}} \text{ inducing this conditional} \Big\},$$

and $K_v^{\text{true}} \subseteq K_v^{\text{local}}$, often strictly: sentences attached to *other* families and the LMC equalities further constrain the conditional but are discarded. `method="nlp"` repairs only one slice of this gap (pairwise parent marginal independence) and is still loose for ≥ 3 parents and for any constraint that spans families.

On the example. The $[0, 1]$ entries of $P(A \mid B{=}0, E{=}1)$ and of $P(C-D \mid A{=}0)$ are pure gap-A artifacts: $s5$ ($P(A) \leq 0.1$) and the parent marginals $s1, s2$ are not seen when family A is solved against only its attached rows, and family $C-D$ never sees that A is rare. The true projections are far narrower.

3.2 (B) Strong-extension widening

Even with $K_v = K_v^{\text{true}}$ exactly, the strong extension

$$\mathcal{D}_{\text{SE}} = \Big\{ \textstyle\prod_v P_v \;:\; P_v \in K_v \text{ chosen independently per parent-config} \Big\}$$

allows every (node, parent-configuration) slot to pick its extreme point freely. The LCN's non-local sentences *couple* choices across families that $\mathcal{D}_{\text{SE}}$ decouples, so $\mathcal{D}_{\text{LCN}} \subsetneq \mathcal{D}_{\text{SE}}$ in general. This is the credal set whose product structure `verify_lmc_factorization.py` already reconstructs (its `_JointReconstructor` takes one extreme point per (node, parent-config) slot and forms the product joint).

On the example. Gap B shows up in the $C-D$ family. The strong extension lets the four states of $P(C{-}D \mid A{=}1)$ be chosen so that $P(C{=}0, D{=}0 \mid A{=}1)$ takes its max 0.7 *and* $P(C{=}1, D{=}1 \mid A{=}1)$ takes *its* max 0.7 in the same joint—a vertex combination whose diagonal sum is 1.4, violating $s4$'s upper bound of 0.7. The LCN forbids that distribution; the strong extension admits it. No tightening of the *intervals* removes it, because the constraint couples two states of the same factor.

Either gap alone suffices to make posteriors loose; a tight scheme must close the one that dominates on the instance at hand.

4 Proposed schemes

D1 — Constraint back-propagation into local sets (attacks A; the new `"linear-tight"` method). When solving family v, replace the constraint source from "sentences attached to v" with "every LCN sentence s whose atom scope satisfies atoms(s) $\subseteq$ scope(v)," and additionally append the LMC bilinear equalities (`lmc_constraint_groups_vec` in `utils/common.py`) restricted to scope(v). This is exactly `method="linear-tight"`: when the enriched constraint set is purely linear the family stays an LP / Charnes–Cooper fractional LP, and when it acquires LMC bilinear equalities it is solved on the existing hardened ipopt (or, under `solver="scip"`, certified-global) path that the removed `"nlp"` option used to drive. Closes the part of gap A expressible within a single family scope; cannot capture constraints spanning atoms in different families. Near-zero structural change—only `_linear_rows_and_obj`'s sentence selection and a scope-restricted LMC row builder.

On the example. Family A's scope is $\{A, B, E\}$, so $s1, s2, s5$ all satisfy atoms(s) $\subseteq$ scope and would now be loaded together with $s3$. The disjunctive $s3$ plus $s5$ ($P(A) \le 0.1$) plus the parent marginals collapse the vacuous $[0, 1]$ rows of $P(A \mid B, E)$ to genuine sub-intervals. Family $C-D$ now sees $s5$, tightening the $A{=}0$ rows. The diagonal-sum coupling of $s4$ is *within* the $C-D$ family scope, so this single change also removes the gap-B vertex combination of Section 3 from that local set.

D2 — Scope merging with a tightness dial (attacks A, partially B; the `merge_budget` knob). Introduce a budget k (`merge_budget`). Merge adjacent families whose combined scope size is $\le k$ into a single *super-family* and compute one joint local credal set over the merged scope (a larger fractional LP / NLP). At k = full scope this is exact inference; at k = single-family it is today's behavior. Provides an anytime tightness-vs-cost knob, with per-super-family cost 2^k. Implemented as a merging pass over `lcn.families` before `ChainGraphFactorization.build`, reusing the existing per-family solver on the larger scope; `estimate_difficulty` can choose k adaptively.

On the example. Merging families A and $C-D$ (they share the edge $A \to C-D$) into a super-family over $\{A, B, E, C, D\}$ makes $s3, s4, s5$ act jointly: the bound on $P(C{=}D \mid A)$ is now solved in a

model that also knows $P(A) \leq 0.1$ and the $A \mid B, E$ structure, so the $C-D$ conditionals tighten beyond what D1 achieves in isolation. Here k=5 is the full scope, i.e. exact; a budget k=3 would merge only where cheap.

D3 — Joint-LP certification of each local bound (attacks A; exact, certified; the `solver="scip"` path). For each (family, interpretation, sense), solve the bound for the family conditional $P(\mathrm{ch}_v \mid \mathrm{pa}_v)$ against the *full* LCN feasible region: all LCN sentences and all LMC equalities over the 2^n joint, with a fractional objective local to the family. This yields K_v^{true} exactly. It is reached by combining `method="linear-tight"` with `solver="scip"` (the existing global path) so the bounds carry a global optimality certificate. Cost is 2^n variables per solve; viable for small n or, more usefully, as the *certifier* of the cheaper schemes (it is essentially `ExactInference` run per conditional).

D4 — Region-restricted (extensive) credal network (attacks B; structural). Replace the free per-parent-config vertex choice with a *coupled* one. Variants: *(4a)* detect sentences that couple multiple families and add constraints in `cve.py`'s bucket elimination forbidding vertex combinations that violate them; *(4b)* introduce a shared latent "regime" parent to each coupled family so the local credal sets become conditional on a common selector, restoring the cross-family correlation the LCN imposes. Touches the CVE elimination, not the local solves.

On the example. D4 directly kills the $s4$ violation of Section 3: the four extreme points of $P(C{-}D \mid A{=}1)$ may no longer be combined into a joint whose diagonal sum exceeds 0.7. Under variant 4a, CVE's bucket for C–D carries the side constraint $P(C{=}0, D{=}0 \mid A{=}1) + P(C{=}1, D{=}1 \mid A{=}1) \leq 0.7$ across the vertex choice, restoring the coupling that the free product had dropped.

Implementation note (what D4 can and cannot couple). The $s4$ example above is, on inspection, an *in-family* constraint ($\mathrm{atoms}(s4) = \{C, D, A\} \subseteq$ the $C-D$ family scope), so D1 already removes it; it is shown only to illustrate the coupling mechanism. The constraints D4 must actually handle are the *cross-family* ones — those whose atom set is a subset of no single family scope. A subtle but important fact constrains the menu: the strong extension of a chain-graph credal network is a *product* $\prod_v P(\mathrm{ch}_v \mid \mathrm{pa}_v)$, and that product structure *satisfies every LMC independence by construction* — which is exactly why `verify_lmc_factorization.py` reports the factorization SUPPORTED on every chain LCN we have tried. Consequently *no* product joint can violate a cross-family *LMC assertion*: enforcing them in D4 is sound but inert. The gap-B looseness D4 removes is therefore carried by cross-family *sentences* (Type-1/Type-2 bounds whose scope spans families), which the free product *can* violate because each local factor is chosen to satisfy only its own attached sentences. As a concrete witness, a three-family instance $P(A), P(B \mid A), P(C \mid A)$ with the cross-family sentence $P(B \wedge C) \leq 0.05$ admits a free-product joint reaching $P(B \wedge C) = 0.92$; D4 forbids it, collapsing e.g. $P(B \mid C{=}1)$ from $[0.04, 0.96]$ to a near-point. The implementation (`cn/coupling.py`, the `coupling="cross-family"` flag on Credal VE / CTE / ApproxLP) classifies a constraint as cross-family against the *built* factors, so a D2 `merge_budget` that fuses the coupled families turns the same constraint in-family and removes it from D4's list — D2 and D4 compose without double-counting.

D5 — Constraint-aware CVE (attacks A and B; the tight target). Keep the chain-graph structure for the elimination order and bucket scopes, but at each bucket combination optimize the bucket potential subject to the original LCN sentences *projected onto the bucket scope*, rather than over the free strong-extension vertices. This is a sequence of small NLPs threaded through variable elimination—a credal VE that respects the global constraints at every step and never materializes

Scheme	Source closed	Tightness	Cost	Effort
D1 back-propagate in-scope constraints	A (partial)	+	≈ 0	low
D2 scope merging (dial k)	A (+ B)	dial	2^k	low–med
D3 joint-LP certified locals	A (exact)	++	2^n/solve	med
D4 region-restricted CVE	B	+	med	med
D5 constraint-aware CVE	A + B	+++ ($\rightarrow$exact)	treewidth	high

Table 1: Summary of the proposed schemes against the two looseness sources of Section 3.

the loose strong extension. In the limit it equals exact inference but its cost is bounded by the largest bucket (treewidth), not 2^n.

On the example. Eliminating B, E first produces a bucket over $\{A\}$ whose potential is optimized subject to $s1, s2, s3, s5$ jointly (closing gap A for A); eliminating C, D uses a bucket over $\{A\}$ optimized subject to $s4$ as a coupled sum (closing gap B for $C-D$). The query marginal $P(A)$ is then read off the final $\{A\}$ bucket. No step ever forms the loose product, and no bucket exceeds three atoms.

Implementation note (junction-tree exact NLP). D5 is realized (`cn/junction_nlp.py`, the `coupling="d5"` flag on Credal VE) not as a sequence of message NLPs but as a single equivalent *junction-tree* NLP, which is cleaner to make provably exact. Build a clique tree from the chain-graph elimination order (reusing the bucket-tree construction), introduce one variable vector per cluster ranging over that cluster's joint distribution, tie adjacent clusters with Lauritzen separator-consistency equalities (the marginal of a cluster onto a separator equals the neighbour's), and impose each LCN sentence and LMC equality on a cluster containing its atoms. Because the clique tree has the running-intersection property, a separator-consistent constraint-satisfying family of cluster marginals projects to exactly the marginals of some global joint in the LCN's set, and conversely; hence min / max of the query marginal over this NLP equals the exact bound, at cost $\sum_c 2^{|\text{atoms}(c)|}$ rather than 2^n. Cross-family constraints (whose atoms span cliques) are handled by augmenting the elimination order with their node sets so they acquire a single host cluster — the JT analogue of D2's family merging and D4's order augmentation; the cluster a constraint forces sets the local treewidth, so on instances whose cross-family constraints are themselves near-global (e.g. `alarm`, whose LMC assertions $C \perp B, E \mid D, A$ span all five atoms) D5 degenerates to near-full-joint cost, while on instances with local cross-family coupling it stays cheap.

Two findings from the implementation. (1) The cluster NLP is nonconvex (Type-2 ratios, the bilinear LMC equalities, and the conditional-query ratio objective), so a local solver (ipopt) only reaches a local optimum that can fall *short* of the true bound; D5 must use the certified global backend (SCIP) to actually be exact, and defaults to it. (2) D5 is exact for *conditional* queries $P(q \mid e)$, where the vertex-enumeration methods (plain Credal VE, and D4 on top of it) are *not even sound*: the conditional bound is a ratio whose extremum need not lie at a vertex of the unconditioned credal set, so min/max-over-vertices can land strictly inside the true interval. On `examples/d4_biting.lcn`, $P(B \mid C{=}1)$ is exactly $[0.006, 0.926]$ (D5, matching the global solver), whereas plain Credal VE reports $[0.038, 0.963]$ and D4 reports the spurious near-point $[0.038, 0.038]$. D5 is therefore the only member of the family that is sound for conditional queries, not merely tighter.

$P(A \mid B, E)$ row	`linear`	`linear-tight` (D1)
A=0, B=0, E=0	[0.956, 1.0]	[0.988, 1.0]
A=1, B=0, E=0	[0, 0.044]	[0, 0.015]
A=0, B=1, E=0	[0, 1.0]	[0, 0.720]
A=1, B=1, E=0	[0, 1.0]	[0.280, 1.0]

Table 2: D1 on `alarm`'s A family: the vacuous $[0, 1]$ rows tighten once the scope-restricted constraints (here $s1, s2, s5$ and the $(B \perp E)$ equality) are imposed on the family joint. Bounds rounded; computed with `method="linear-tight"`.

5 Detailed algorithms and worked examples

This section gives a step-by-step description of each scheme as implemented in the `cn/` package, together with a fully worked example on a running instance and a figure that traces the algorithm's state. Two running instances are used:

- `alarm` (Section 2): five atoms $\{A, B, C, D, E\}$, the compound node C–D, families $P(A \mid B, E)$, $P(B)$, $P(E)$, $P(C-D \mid A)$. Used for D1, D2, D3 (local-set tightening, gap A).
- `d4_biting` (`examples/d4_biting.lcn`): three atoms, families $P(A)$, $P(B \mid A)$, $P(C \mid A)$ and the *cross-family* sentence $P(B \wedge C) \leq 0.05$. Used for D4 and D5 (cross-family coupling, gap B), because `alarm` has no cross-family *sentence* and so leaves gap B inert.

Throughout, an interpretation of a scope $X_1, \ldots, X_k$ is packed *MSB-first*: the integer index of $(x_1, \ldots, x_k)$ is $\sum_i x_i \, 2^{k-1-i}$. A local credal set for a factor $P(\text{ch} \mid \text{pa})$ is represented by its interval bounds $[\underline{p}, \overline{p}]$ per (child-state, parent-config); its *extreme points* are the finite set of conditional vectors realising those bounds, enumerated by the credal-net compiler (`cnv.extreme_points`).

5.1 D1 — constraint back-propagation (`method="linear-tight"`)

What it computes. For one family v with flattened atom scope $\text{sc}(v) = \text{ch}_v \cup \text{pa}_v$ and one interpretation π of that scope, D1 returns the lower/upper bound of the conditional $P(\text{ch}_v{=}c \mid \text{pa}_v{=}p)$ optimised not over the family's attached sentences alone but over *all* sentences whose atoms lie in $\text{sc}(v)$ *and* the LMC independence equalities whose atoms lie in $\text{sc}(v)$. The pipeline already attaches every in-scope *sentence* to a family (via `process_chain_graph`), so the only delta over `method="linear"` is the addition of the scope-restricted *LMC bilinear equalities*; when a family has no in-scope LMC assertion, D1 falls back to the plain linear program and equals `"linear"`.

Worked example (`alarm`, family $P(A \mid B, E)$). The scope is $\{A, B, E\}$. The in-scope sentences are $s1 : P(B) \in [0.1, 0.2]$, $s2 : P(E) \in [0.05, 0.1]$, $s3 : P(A \mid B \vee E) \in [0.6, 0.9]$, $s5 : P(A) \in [0.05, 0.1]$. The one in-scope LMC assertion is $(B \perp E)$, contributing the *marginal* equality $P(B{=}1, E{=}1) = P(B{=}1)\,P(E{=}1)$ over the $\{A, B, E\}$ joint. Table 2 shows the effect: the rows that were vacuous $[0, 1]$ under `"linear"` (because $s3$ only constrains the $B \vee E$ corner) collapse to genuine sub-intervals once $(B \perp E)$ plus $s1, s2, s5$ act jointly on the family's own joint. Figure 1 sketches the model.

5.2 D2 — scope merging with budget k (`merge_budget`)

What it computes. D2 is a *pre-factorization* pass: before the local credal sets are solved, it merges adjacent families into super-families whose flattened scope is at most k atoms, then runs D1

Algorithm 1 D1 local credal set for one family (`_build_linear_tight_model`)

Require: family scope sc $= [s_1, \ldots, s_k]$; attached sentences S; LCN independencies $\mathcal{I}$; child state c, parent config p; sense $\in \{\min, \max\}$

$\mathcal{A} \leftarrow \{\, a \in \mathcal{I} : \text{atoms}(a) \subseteq \text{sc} \,\}$ ▷ in-scope LMC assertions

if $\mathcal{A} = \varnothing$ **then return** SolveLinear(sc, $S, c, p,$ sense) ▷ pure LP / Charnes–Cooper, identical to `"linear"`

end if

enumerate the $N = 2^k$ interpretations of sc; create variables $q_0, \ldots, q_{N-1} \geq 0$ ▷ a joint over the family scope

add the simplex constraint $\sum_i q_i = 1$

for all sentences $s \in S$ **do**

  if s is Type-1 $\ell \leq P(\varphi) \leq u$ **then** add $\ell \leq \mathbf{1}_{\varphi}^{\top} q \leq u$

  else (s is Type-2 $\ell \leq P(\varphi \mid \psi) \leq u$) add $\mathbf{1}_{\varphi\psi}^{\top} q \geq \ell\, \mathbf{1}_{\psi}^{\top} q$ and $\mathbf{1}_{\varphi\psi}^{\top} q \leq u\, \mathbf{1}_{\psi}^{\top} q$

  end if

end for

for all $a = (X \perp Y \mid Z) \in \mathcal{A}$ and each group of LmcGroups$(a,$ sc$)$ **do**

  if conditional group $(\mathbf{1}_{xyz}, \mathbf{1}_{z}, \mathbf{1}_{xz}, \mathbf{1}_{yz})$ **then** add $(\mathbf{1}_{xyz}^{\top} q)(\mathbf{1}_{z}^{\top} q) = (\mathbf{1}_{xz}^{\top} q)(\mathbf{1}_{yz}^{\top} q)$ ▷ bilinear

  else (marginal group) add $\mathbf{1}_{xy}^{\top} q = (\mathbf{1}_{x}^{\top} q)(\mathbf{1}_{y}^{\top} q)$

  end if

end for

build the objective $P(\text{ch}{=}c \mid \text{pa}{=}p) = (\mathbf{1}_{c,p}^{\top} q)/(\mathbf{1}_{p}^{\top} q)$ via the aux variable $t\,(\mathbf{1}_{p}^{\top} q) = \mathbf{1}_{c,p}^{\top} q,\ t \in [0,1]$

return Optimise(sense, t) with hardened ipopt (multi-restart) or, under `solver="scip"` (this is **D3**), the certified global solver

(or `"linear"`) on the larger scopes. Because a merged scope is a superset of each member scope, more sentences and more LMC assertions become in-scope, so the local sets can only tighten. At k=1 no merge happens (identical to today); at $k \geq n$ everything collapses into one family (exact). D2 operates on a *copy* of `lcn.families` and never mutates the model.

Worked example (`alarm`, k=5). The families are $P(A \mid B, E)$ (scope $\{A, B, E\}$), $P(B)$, $P(E)$, $P(C - D \mid A)$ (scope $\{C, D, A\}$). With k=5 the greedy pass contracts the arc $A \to (C - D)$ (their union $\{A, B, E, C, D\}$ has size $5 \leq k$), then folds B and E into the same super-family along the arcs $B \to A$, $E \to A$, producing a single family over all five atoms (Figure 2). Now *every* sentence and *every* LMC assertion is in-scope, including the cross-family assertions $(C \perp B, E \mid D, A)$ and $(D \perp B, E \mid C, A)$ that no 3-atom family could host — so the k=5 local set is the exact joint and the $C{-}D$ conditionals tighten beyond what D1 achieves on the 3-atom family. A budget k=3 would leave the families unchanged here (every pairwise union already exceeds 3), recovering plain D1.

5.3 D3 — certified local bounds (`solver="scip"`)

What it computes. D3 is not a separate model but a *backend*: it solves the D1 (or D2) local program with SCIP's spatial branch-and-bound global solver instead of the local ipopt. The D1 program is nonconvex (Type-2 ratio constraints and the bilinear LMC equalities), so ipopt only returns a *locally* optimal bound, which can be strictly looser *or* (for a feasibility-style extreme) spuriously tighter than the true local bound. SCIP returns the bound with a global optimality

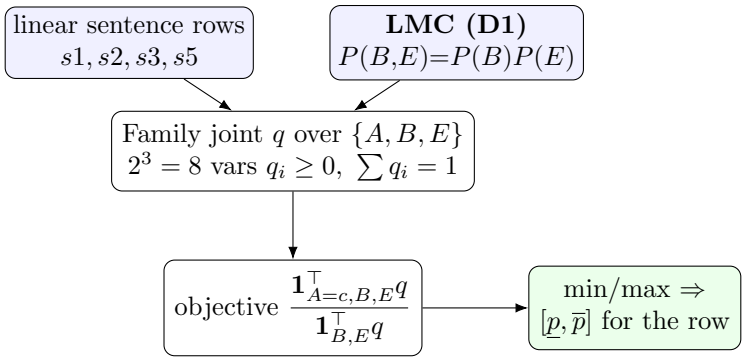

Figure 1: D1 (Algorithm 1). The family-scope joint q is constrained by the in-scope sentences *and* the scope-restricted LMC equalities; the conditional bound is read off as a fractional objective. `"linear"` omits the blue LMC block.

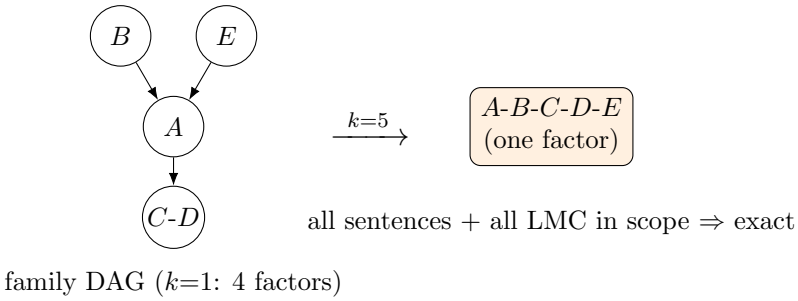

Figure 2: D2 (Algorithm 2) on `alarm` with k=5: greedy arc contraction collapses the four families into one super-family over $\{A, B, C, D, E\}$, bringing the cross-family LMC assertions in-scope. At k=1 the left DAG is used unchanged.

certificate (or a bounded gap under a time limit). D3 thus serves two roles: it produces the exact local credal set K_v^{true} for a family scope, and it *certifies* the cheaper D1/ipopt bounds.

Worked example (`alarm`, certifying $P(C$–$D \mid A)$). On the diagonal state $P(C{=}0, D{=}0 \mid A{=}1)$ the isolated family LP gives $[0, 0.7]$ (only the sum $P(C{=}D \mid A) \leq 0.7$ of $s4$ binds). Running the same model under `solver="scip"` returns the identical interval *with a zero optimality gap*, certifying that 0.7 is the true family-projected upper bound and that the ipopt result was not a local artifact. The contract D3 enforces is the containment

$$[\underline{p}^{\star},\ \overline{p}^{\star}]_{\text{D3/scip}} \ \subseteq\ [\underline{p},\ \overline{p}]_{\text{D1/ipopt}} \ \subseteq\ [\underline{p},\ \overline{p}]_{\text{linear}},$$

checked per (family, row): a violation of the left inclusion flags an unsound ipopt local optimum, the width gap on the right quantifies how much looseness D1 still leaves (gap A residual). Figure 3 shows the role.

5.4 D4 — region-restricted coupling (`coupling="cross-family"`)

What it computes. Credal VE represents each node's local credal set as a `Potential`: a finite set of *functions* (numpy arrays), each one a fully instantiated CPT obtained by picking, independently per parent-configuration, one local extreme point. Bucket elimination repeatedly *combines*

Algorithm 2 D2 family merging (`_merge_families`)

Require: families $\mathcal{F} = \{(\mathrm{ch}_i, \mathrm{pa}_i, S_i)\}$; budget k

build the *family DAG*: a node per family child; an arc $u \to v$ whenever ch_u is a parent of v
repeat
  for all arcs (u, v) in deterministic (sorted) order **do**
    $\mathrm{sc} \leftarrow \mathrm{flatten}(\mathrm{scope}(u) \cup \mathrm{scope}(v))$
    if $|\mathrm{sc}| > k$ **then continue**
    end if
    form the super-family M: child atoms $= \mathrm{ch}_u \cup \mathrm{ch}_v$ (node name `"-".join(sorted(...))`); parents = external parents only (drop parents produced inside M)
    rewire every family that had u or v as a parent to point at M
    if the rewired family DAG is acyclic **then** accept the contraction; **restart** the scan
    else reject (keep searching) ▷ D2 never creates a cycle
    end if
  end for
until no contraction was accepted
for all resulting (super-)families M **do**
  recompute attached sentences $S_M = \{s : \mathrm{atoms}(s) \subseteq \mathrm{scope}(M)\}$ ▷ can only grow
end for
return the merged family list (fed to D1/`"linear"` on the larger scopes)

Algorithm 3 D3 certified local bound (`solver="scip"` on the D1 model)

Require: the D1 model $\mathcal{M}$ for (family, row, sense) from Algorithm 1

add the denominator floor $\mathbf{1}_p^\top q \geq \varepsilon$ (so the conditional ratio stays well defined and SCIP cannot drive $P(\mathrm{pa}{=}p) \to 0$)
$(\mathrm{val}, \mathrm{gap}, \mathrm{status}) \leftarrow \textsc{ScipSolve}(\mathcal{M},\ \mathrm{time\_limit},\ \mathrm{gap\_tol})$
if status $\in \{\textsc{optimal}\}$ **or** gap $\leq$ gap_tol **then return** certified val
else return best-so-far val with the reported gap ▷ partial certificate
end if

potentials (a Cartesian product of their function sets) and *marginalises* a variable out; the final query bound is the min / max of the normalised query state over the surviving functions. This is the free strong-extension product. D4 *drops* any function whose assembled joint violates a *cross-family* constraint — a sentence or LMC assertion whose atom set is a subset of no single family scope. Crucially, a fully assembled function over a node set *is* the unnormalised joint over those nodes' atoms, so a cross-family residual can be evaluated on it directly, reusing exactly the verifier's residual math.

The two pieces that make it correct.

1. **Checkability gate.** A constraint is evaluated on a function only once the function's scope covers *all* the constraint's atoms; until then the predicate returns "feasible" (do not drop). This makes filtering after *any* elimination step sound: it can only ever remove a genuinely infeasible function, never one whose feasibility is undecided.

2. **Order augmentation.** For the gate to ever fire, the constrained atoms must co-occur in some bucket. D4 therefore forces the min-fill elimination order, augmented with one extra

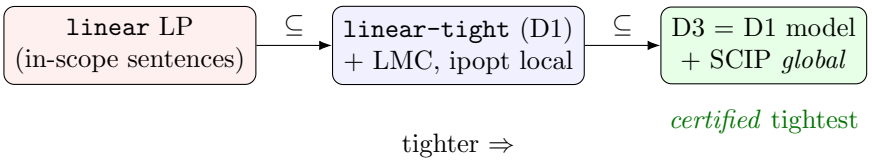

Figure 3: D3 (Algorithm 3) solves the D1 model with the global solver, certifying the local bound. The nesting D3 ⊆ D1 ⊆ `linear` is the per-row validation contract (Section 6).

Algorithm 4 D4 coupled Credal VE (`coupling="cross-family"`)

Require: credal net *cnv*; query q; evidence e

$\mathcal{C} \leftarrow$ cross-family constraints $\{c : \text{atoms}(c) \not\subseteq \text{scope}(f)\ \forall \text{ families } f\}$
if $\mathcal{C} = \varnothing$ **then** run plain Credal VE ▷ D4 inert; e.g. `alarm`
end if
order $\leftarrow$ MinFill(factor scopes $\cup$ { node set of $c : c \in \mathcal{C}$}, exclude=$\{q\}$)
build one `Potential` per node from its extreme points; add evidence indicators
for all v in *order* **do**
    bucket $\leftarrow$ potentials whose scope contains v; *combined* $\leftarrow$ product of the bucket
    D4 filter: keep $f \in$ *combined* iff Feasible(f, scope(*combined*), $\mathcal{C}$) ▷ on the full bucket scope, *before* marginalising
    result $\leftarrow$ Marginalise(*combined*, v); *result* $\leftarrow$ Prune(*result*)
end for
combine the leftovers; **return** min / max of the normalised q state over the surviving functions

function Feasible(f, sc, $\mathcal{C}$)
    for all $c \in \mathcal{C}$ with atoms(c) $\subseteq$ sc **do**
        marginalise f to atoms(c), normalise, evaluate the residual (Type-1/2 bound or bilinear LMC); **if** violated **return** false
    end for
    return true ▷ atoms not yet all in scope ⇒ not droppable
end function

"clique scope" per cross-family constraint (its node set), so those nodes land in a common bucket — otherwise an atom is summed out before the constraint scope assembles and D4 stays inert (sound but useless).

Worked example (`d4_biting`, query $P(B \mid C{=}1)$). The cross-family sentence is $P(B \wedge C) \leq 0.05$ (atoms $\{B, C\}$, fitting neither the B nor the C family). The free product can pick $P(B{=}1 \mid A)$ and $P(C{=}1 \mid A)$ both large, giving a joint with $P(B \wedge C) = 0.92$ — admissible in the strong extension but forbidden by the LCN. Augmenting the order forces $\{A, B, C\}$ into one bucket; the D4 filter there drops every function whose $P(B \wedge C)$ exceeds 0.05. Figure 4 traces it. Net effect on the *unconditional* marginals is small, but on a query that makes B and C interact (e.g. evidence $C{=}1$) the upper bound on $P(B \mid C{=}1)$ drops sharply.

A sharp limitation (conditional queries). D4 is sound only as a tightening of the strong-extension bound, and the strong-extension bound is itself a valid *outer* bound only for *unconditional* marginals. For a conditional query $P(q \mid e)$ the bound is a ratio whose extremum need not lie at a

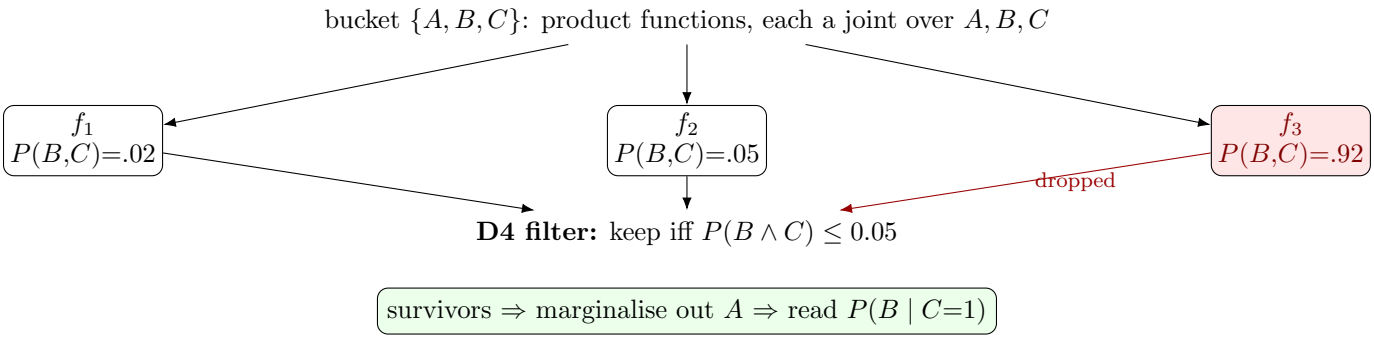

Figure 4: D4 (Algorithm 4) on `d4_biting`: in the augmented bucket $\{A, B, C\}$, functions whose joint violates the cross-family $P(B \wedge C) \leq 0.05$ (e.g. f_3 with 0.92) are dropped *before* A is summed out; the survivors yield the tightened query.

vertex of the unconditioned credal set, so min / max-over-vertices — and hence D4's filtered version — can land *strictly inside* the true interval. On `d4_biting`, $P(B \mid C{=}1)$ is truly $[0.006, 0.926]$, yet plain Credal VE reports $[0.038, 0.963]$ and D4 the spurious near-point $[0.038, 0.038]$. The implementation prints a warning when `coupling="cross-family"` is used with evidence and directs the user to D5 (Section 5.5), which is exact.

5.5 D5 — junction-tree exact NLP (`coupling="d5"`)

What it computes. D5 abandons the vertex product entirely and computes the *exact* chain-graph marginal by a single nonlinear program over *cluster-marginal* variables. Build a clique (junction) tree from the chain-graph elimination order; give each cluster c a variable vector q_c ranging over the joint distribution of its atoms; tie adjacent clusters with Lauritzen *separator-consistency* equalities (the marginal of one cluster onto a shared separator equals the neighbour's); and impose every LCN sentence and LMC equality on a cluster whose atom set contains it. Reading the query marginal off the cluster that holds the query (and evidence) and optimising it min / max over this feasible region returns the exact LCN bound. Because each cluster is bounded by the treewidth, the NLP has $\sum_c 2^{|\text{atoms}(c)|}$ variables rather than 2^n.

Why it is exact. For a chain graph the clique tree has the *running intersection property* (RIP). With RIP, a family of cluster marginals that (i) agrees on every separator and (ii) satisfies, in some containing cluster, every LCN/LMC constraint, extends to a single global joint feasible for the full 2^n `ExactInference` model — and conversely every feasible global joint projects to such a family. Hence the cluster NLP and the full-joint NLP have the same optimal value. Cross-family constraints (and the query+evidence atoms) are forced to share a host cluster by augmenting the elimination order with their node sets, exactly as in D4; if that would push a cluster past the treewidth budget, D5 falls back to `ExactInference.run_query` (still exact, at 2^n).

Worked example (`d4_biting`, query $P(B \mid C{=}1)$). The cross-family sentence $P(B \wedge C) \leq 0.05$ has node set $\{B, C\}$, and the query+evidence atoms are $\{B, C\}$; augmenting forces them together, so the elimination yields a clique tree with a cluster $c_q{=}\{A, B, C\}$ (the full joint) and a subsumed leaf that is pruned. All four sentences, the LMC assertion $(B \perp C \mid A)$, and the objective all host on c_q. The NLP is therefore identical to the full-joint `ExactInference` model, and solving it with SCIP returns $P(B \mid C{=}1) = [0.006, 0.926]$ — the exact bound, matching the certified global oracle

Algorithm 5 D5 junction-tree exact NLP (`build_and_solve_jt_nlp`)

Require: credal net cnv; query q; evidence e; budget $k_{\max}$; solver

$\mathcal{C} \leftarrow$ cross-family constraints; augment the min-fill order with their node sets and the query+evidence node set

build the clique tree T (clusters $\mathcal{K}$, edges with separators) from the augmented order; flatten each cluster to its atom set; prune leaf clusters subsumed by a neighbour

if $\max_c |\text{atoms}(c)| > k_{\max}$ **then return** ExactInference(q, e) ▷ budget fallback

end if

create variables $q_c[i] \geq 0$ for each cluster c and state $i \in [0, 2^{|\text{atoms}(c)|})$; add per-cluster simplex $\sum_i q_c[i] = 1$

for all tree edges (c, d) with separator S **do**

    build 0/1 marginalisation matrices $M_{c\to S}, M_{d\to S}$; add $M_{c\to S}\, q_c = M_{d\to S}\, q_d$ ▷ separator consistency

end for

for all sentences / LMC assertions **do** assign each to a host cluster containing its atoms; add its row(s) (Type-1/2 linear/ratio, or bilinear LMC) on that cluster's q_c

end for

on the host cluster c_q of $q \cup e$, build the objective $P(q{=}1 \mid e) = (\mathbf{1}_{q,e}^\top q_{c_q})/(\mathbf{1}_e^\top q_{c_q})$ via the aux-variable trick (with denominator floor under SCIP)

$\underline{q} \leftarrow$ Solve(min), $\overline{q} \leftarrow$ Solve(max) with the global backend (SCIP); **return** $[\underline{q}, \overline{q}]$

and correcting both plain Credal VE ($[0.038, 0.963]$) and D4 ($[0.038, 0.038]$). Figure 5 shows the cluster tree and the NLP it induces.

Two implementation findings. (1) The cluster NLP is nonconvex (Type-2 ratios, bilinear LMC, the conditional-ratio objective); the local solver ipopt returns a local optimum that can fall *short* of the bound (it gave 0.90 instead of 0.926 above), so D5 defaults to the certified global backend SCIP. (2) The treewidth saving is real only when the cross-family constraints are themselves local: on `alarm` the assertions $(C \perp B, E \mid D, A)$ and $(D \perp B, E \mid C, A)$ span all five atoms, so the augmentation forces a near-full-joint cluster and D5 degenerates to `ExactInference` cost there.

The `cn/` package ships five inference algorithms beyond CVE. Crucially, *all of them consume the same object*: a built `CredalNetworkVertices`, i.e. the directed structure plus the enumerated extreme points of the local credal sets (`cnv.extreme_points`). They differ only in how they propagate those vertices:

Algorithm	Propagation	Bound character vs. the strong extension
Credal VE (`cve.py`)	per-target bucket elimination	exact (the strong-extension interval)
Credal CTE (`ccte.py`)	two-pass bucket tree	exact (ϵ=0); outer if ϵ>0
ApproxLP (`approxlp.py`)	per-variable LP over vertices	*inner* (subset of the true interval)
Interval BP (`ibp.py`)	loopy interval messages	*outer* (`interval`); inner (`variational`)
Credal IJGP (`ijgp.py`)	join-graph, i-bound	outer-style; tunable by i-bound

(`CredalVE.run(evidence)` now computes *all* singleton-atom marginals in one call by looping the single-query bucket elimination over the credal-network nodes: for each target node it recomputes the elimination order so the target is eliminated last, runs one pass, and reads off the node's

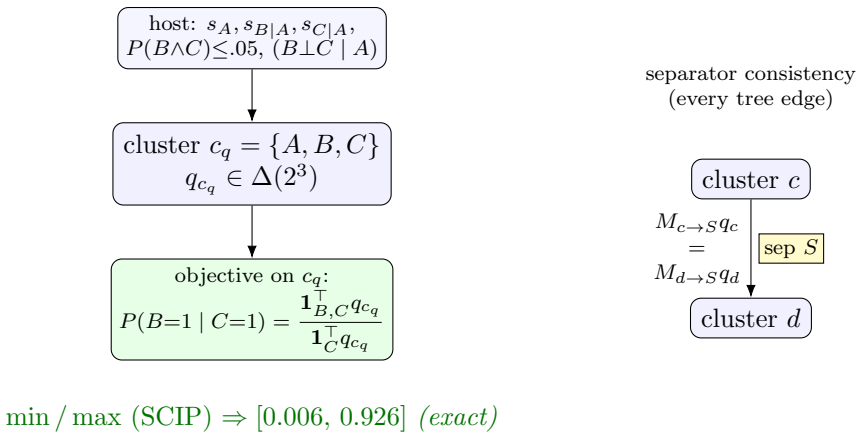

Figure 5: D5 (Algorithm 5) on `d4_biting`: the augmented clique tree has the full-joint cluster $c_q = \{A, B, C\}$ hosting all constraints and the conditional objective; separator-consistency equalities (right) tie clusters on general instances. Solving with SCIP gives the exact $P(B \mid C{=}1)$.

bounds, projecting compound nodes to their atoms by an LP. Because each target gets its own optimal per-target order, its strong-extension bound is at least as tight as Credal CTE's shared two-pass schedule.)

The essential observation is that every one of these algorithms brackets the *strong-extension* interval $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$, not the LCN interval $[\underline{q}_{\mathrm{LCN}}, \overline{q}_{\mathrm{LCN}}]$. Since

$$[\underline{q}_{\mathrm{LCN}}, \overline{q}_{\mathrm{LCN}}] \subseteq [\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}],$$

shrinking the strong extension toward $\mathcal{D}_{\mathrm{LCN}}$ pulls every algorithm's output inward. This splits the schemes into two leverage classes.

Class 1 — input-level tightening (D1, D2, D3): free for all five algorithms. D1/D2/D3 change only what `CredalNetworkVertices.from_lcn` produces—tighter local intervals, hence tighter enumerated vertices. Because the algorithms read those vertices without assuming how they were obtained, *no algorithm code changes*. The effects compose with each algorithm's own character:

- **CVE / CCTE** (exact over the strong extension): their interval is exactly $[\underline{q}_{\mathrm{SE}}, \overline{q}_{\mathrm{SE}}]$, so a tighter strong extension is reported verbatim—the cleanest beneficiary.
- **ApproxLP** (inner): its interval sits *inside* the strong-extension interval. Tighter vertices raise its lower bound and lower its upper bound too, but the inner gap remains; pairing ApproxLP (inner) with CVE (outer) on the *same* tightened vertices brackets the LCN truth from both sides.
- **Interval BP** (outer): tighter local messages mean tighter fixed-point intervals; the loopy outer slack is unchanged but starts from a narrower base.
- **IJGP**: tighter cluster potentials propagate through the join graph; the i-bound still trades cost for tightness independently.

On the example, replacing the vacuous $[0, 1]$ rows (D1) with genuine sub-intervals narrows the vertex sets fed to *all five*, so e.g. a query $P(A)$ tightens for CVE, ApproxLP, IBP, CTE and IJGP simultaneously, with one build-stage change.

Class 2 — propagation-level coupling (D4, D5): per-algorithm work. D4/D5 do not merely narrow intervals; they remove vertex *combinations* (D4) or avoid forming the product altogether (D5). This is a change to the propagation, so each algorithm needs its own adaptation:

- **CVE / CCTE**: add the cross-vertex side constraints (D4) to the bucket / cluster combination step. This is the most natural fit—both already optimize over vertex tuples per bucket, so a coupling constraint slots in directly. D5 is essentially a constraint-aware rewrite of CVE's bucket optimization.
- **ApproxLP**: its per-variable LP already optimizes over a vertex polytope; D4's coupling constraints can be added as extra LP rows, keeping the inner-bound guarantee while shrinking the polytope.
- **Interval BP / IJGP**: message passing does not carry global cross-factor constraints naturally. The practical route is to let D4's coupling be absorbed at the *build* stage where possible (so it reduces to Class 1 for these two), and accept that the residual coupling D4 cannot localize is simply not captured by loopy / join-graph propagation—these remain the loosest of the family, by design.

Practical recommendation. Because Class 1 is free for all algorithms, apply D1 (and D2 where affordable) at the build stage *first*; every downstream algorithm benefits without modification. Reserve the per-algorithm D4/D5 work for CVE/CCTE (and ApproxLP), which can absorb the coupling exactly, and treat IBP/IJGP as fast outer approximations that ride on the Class-1 tightening.

6 Validation strategy

Tightening must be checked against ground truth on two fronts, matching the two gaps.

Local-set tightness (gap A) via D3. D3 is itself the certifier: for any candidate local credal set produced by D1 or D2, compare its interval $[\underline{p}, \overline{p}]$ for each (family, interpretation) against the D3 certified-global interval $[\underline{p}^\star, \overline{p}^\star]$ obtained from the SCIP joint-LP. A correct outer approximation requires $[\underline{p}^\star, \overline{p}^\star] \subseteq [\underline{p}, \overline{p}]$, and the width difference quantifies the residual local looseness D1/D2 leave behind. Because D3 uses the global `solver="scip"` path, these reference intervals are certified, not merely local optima—unlike `ExactInference`, which is only a local solver and is not a reliable oracle.

Extension tightness (gap B) via `verify_lmc_factorization.py`. The factorization verifier reconstructs each product joint from a choice of local extreme points and evaluates the LMC residual

$$P(x, y, s)\, P(s) - P(x, s)\, P(y, s) \;\approx\; 0$$

for every assertion (its `lmc_constraint_groups_vec` residuals). A residual that stays $\leq$ `tol` after a tightening change confirms the scheme has not broken the independence structure while shrinking the set; a non-trivial residual that the LCN does *not* permit localizes a strong-extension (gap B) violation that D4/D5 must address. This check is solver-free and deterministic, so it is a stable regression gate.

End-to-end. On small instances, posterior marginals from CVE under each scheme are compared to the D3-style full-joint bounds for the query atom (the same certified global solve, with the query as objective). The recommended sequence is D1, then D2 with D3 as the per-instance certifier of gap A; D4/D5 are pursued only once the D3 comparison shows the residual gap is genuinely the strong-extension gap B rather than remaining local looseness.